
Implementation - Continuous progress review

Student Names :

Harsh Doshi(92200133002)

Krish Mamtora (92200133022)

Rishit Rathod(92200133027)

Problem Statement:

Students, parents, and faculty face challenges in tracking and understanding a student's overall performance, including academics, co-curricular, and extra-curricular activities. Currently, there is no single system to calculate and compare scores easily, track activities, or give clear insights into strengths and weaknesses. This makes it hard for students to improve, for parents to follow progress, and for faculty to manage evaluations efficiently.

Objectives:

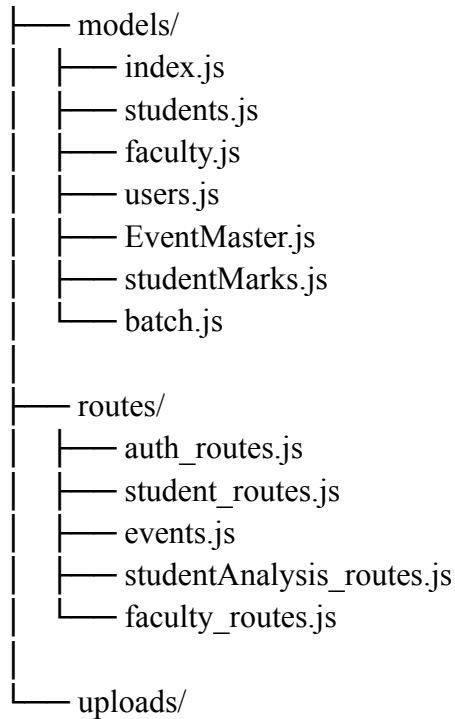
The main goal of this project is to build a single platform where academic performance and activities can be tracked together, including mapping to Course Outcomes (COs) and Bloom's Taxonomy. It will automate score calculations to save time and avoid mistakes. The system will provide easy-to-understand dashboards for students and parents to view grades, rankings, and activity records. For faculty and Heads of Departments, it will offer detailed analytics to help make better decisions about teaching and student performance.

Technical Documentation:

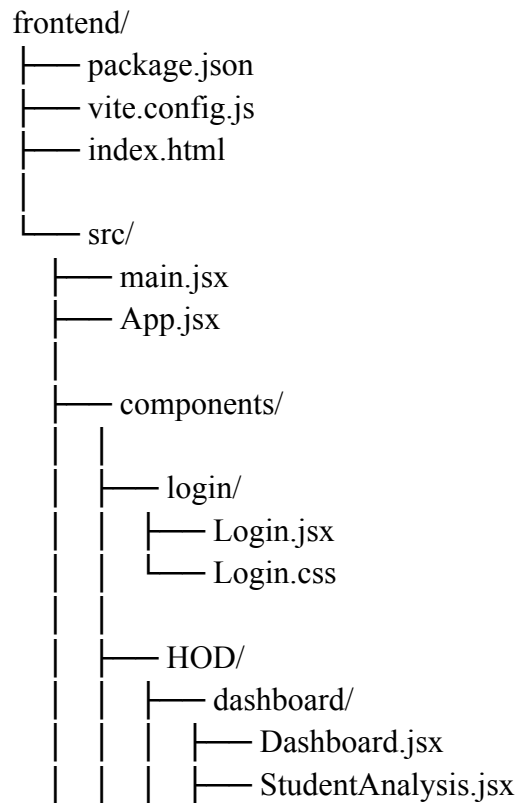
1. Modular code structure

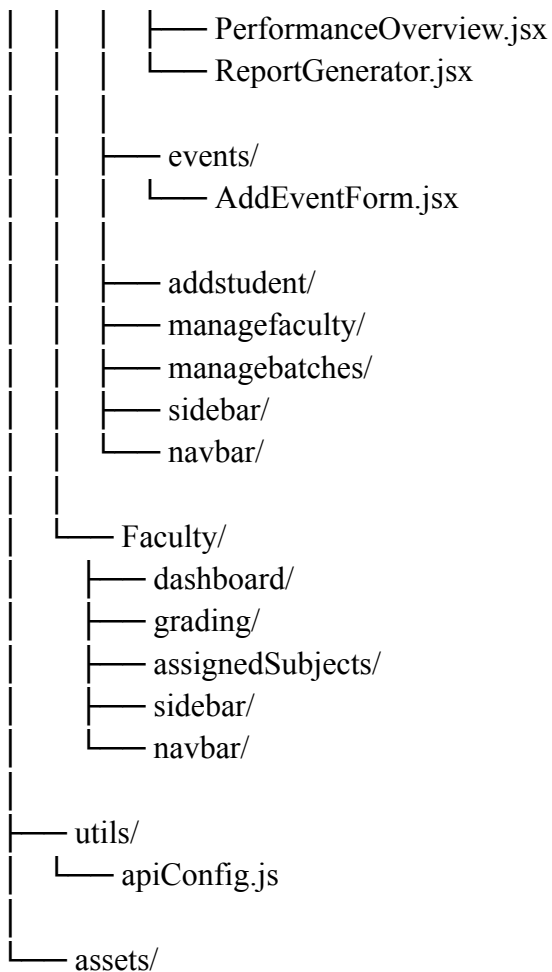
Backend folder structure

```
backend/
├── package.json
├── server.js
├── app.js
├── controller/
│   ├── authController.js
│   ├── studentController.js
│   ├── eventController.js
│   ├── facultyController.js
│   ├── studentAnalysisController.js
│   └── excelUploadController.js
```



Frontend folder structure





2. Module Description

Frontend Modules:

1. HOD Dashboard Module (/components/HOD/)
 - Purpose: admin interface for Heads of Department
 - Components: Student management, faculty assignment, performance analytics
2. Faculty Interface Module (/components/Faculty/)
 - Purpose: Grade entry and subject management for faculty members
 - Components: Grade input forms, subject allocation
3. Authentication Module (/components/auth/)
 - Purpose: User authentication and role-based access control
 - Key Components: Login forms, protected routes, session management

Backend Modules

1. Controller Layer
 - studentAnalysisController.js: Handles student performance analytics and Bloom's taxonomy analysis
 - studentMarksController.js: Manages grade entry, validation, and retrieval
 - eventController.js: Processes event management and student participation
 - authController.js: Handles user authentication and authorization
2. Route Layer
 - RESTful API design with organized endpoints
 - Modular routing for different functionalities
 - Middleware integration for authentication and validation
3. Model Layer
 - Sequelize ORM for database abstraction
 - Complex relationships between entities
 - Data validation and constraints

3. Technology used

1. Frontend (React.js)

- Framework: React 19.0.0 with Vite build tool
- Routing: React Router DOM for navigation
- State Management: React Hooks (useState, useEffect)
- UI Components: Custom components with Lucide React icons
- Charts & Visualizations: Chart.js and Recharts
- Styling: CSS modules with responsive design

2. Backend (Node.js / Express)

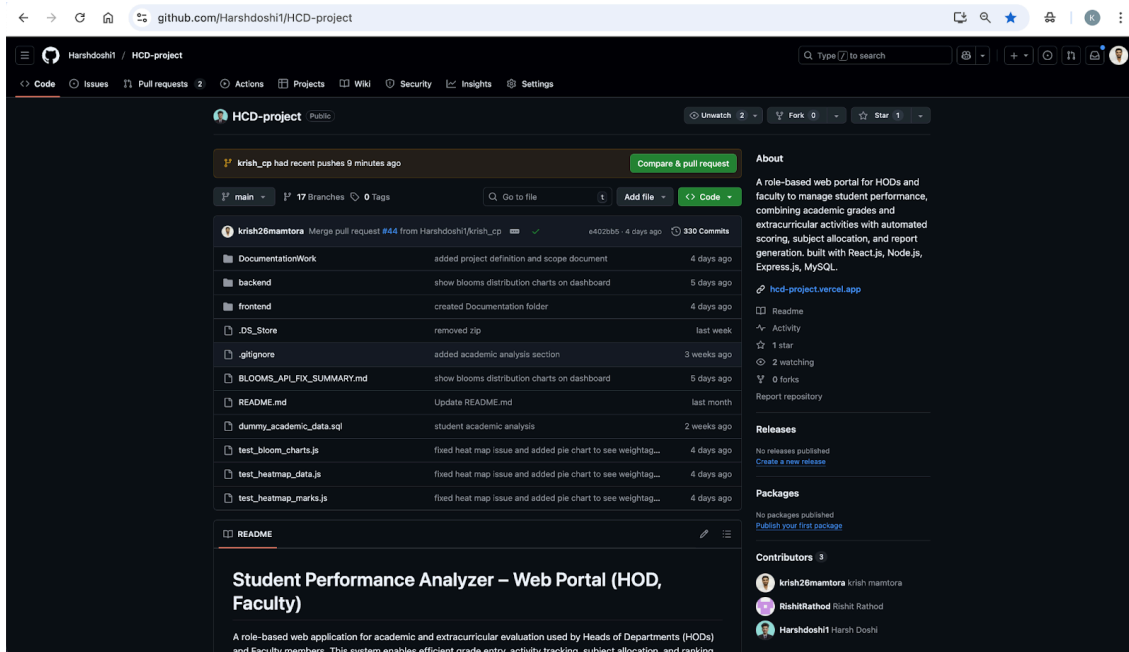
- Framework: Express.js 4.21.2
- Database: MySQL with Sequelize ORM 6.37.5
- Authentication: JWT (JSON Web Tokens) with bcrypt for hashing
- File Processing: Multer (uploads), XLSX (Excel file handling)
- Email: Nodemailer for notifications
- Security: CORS enabled, password encryption with bcrypt

Testing done:

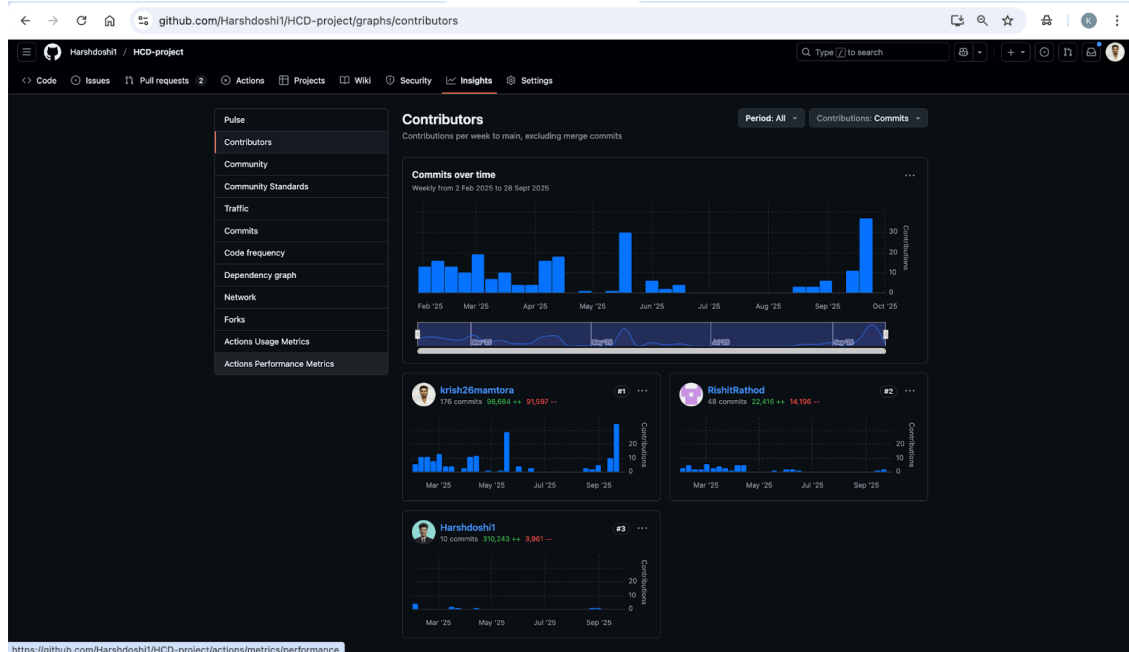
Unit tests: Testing individual components and functions in isolation

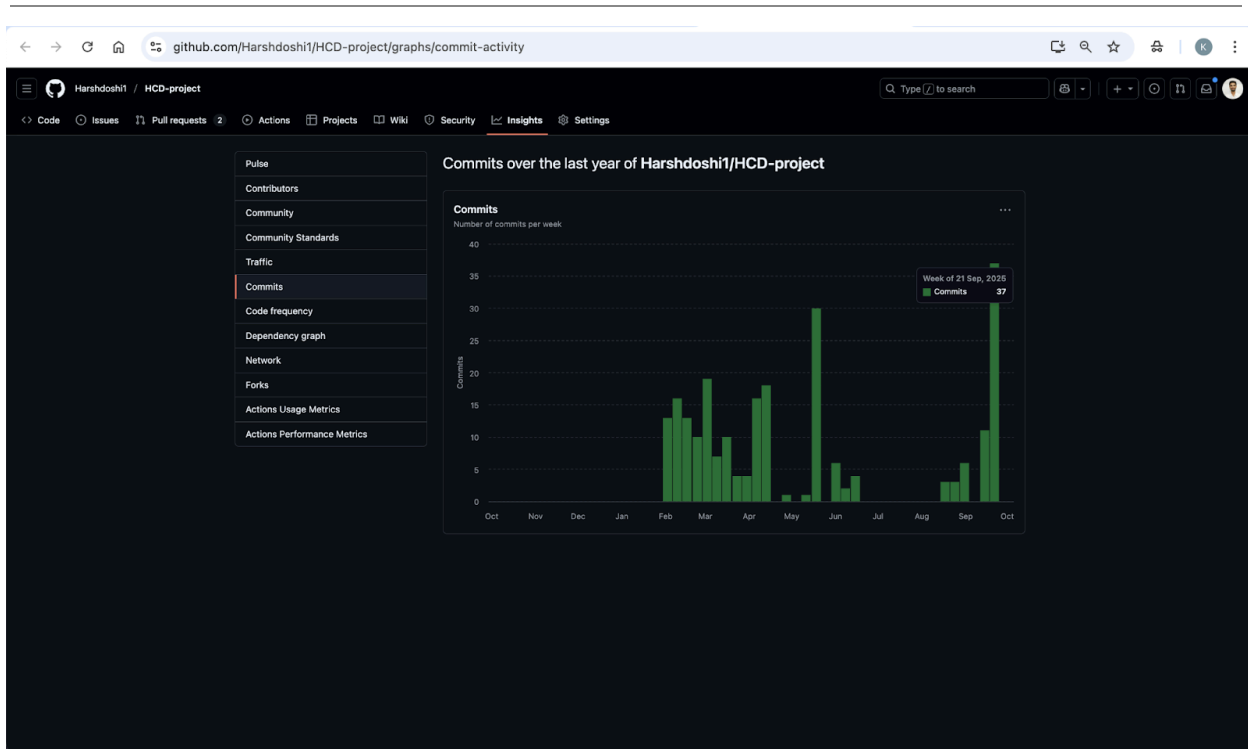
Integration tests: Testing component interactions and API endpoints

Github repository , contributions and commit history for Website Repository

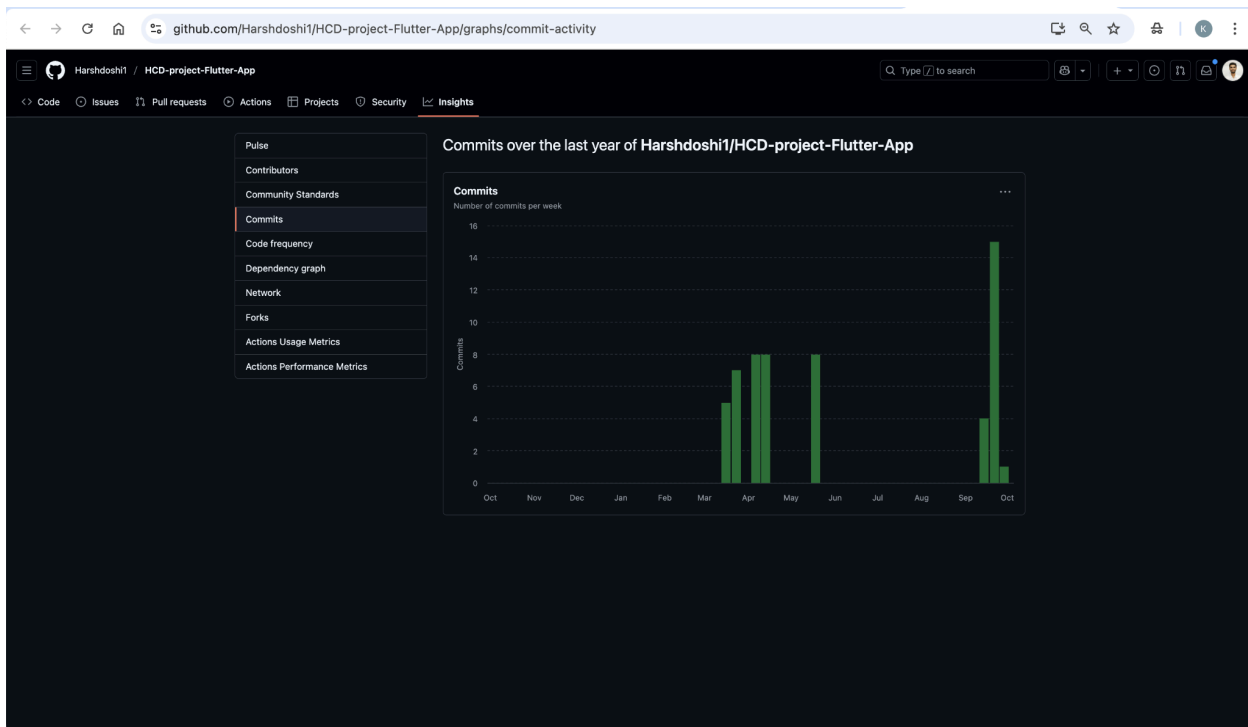


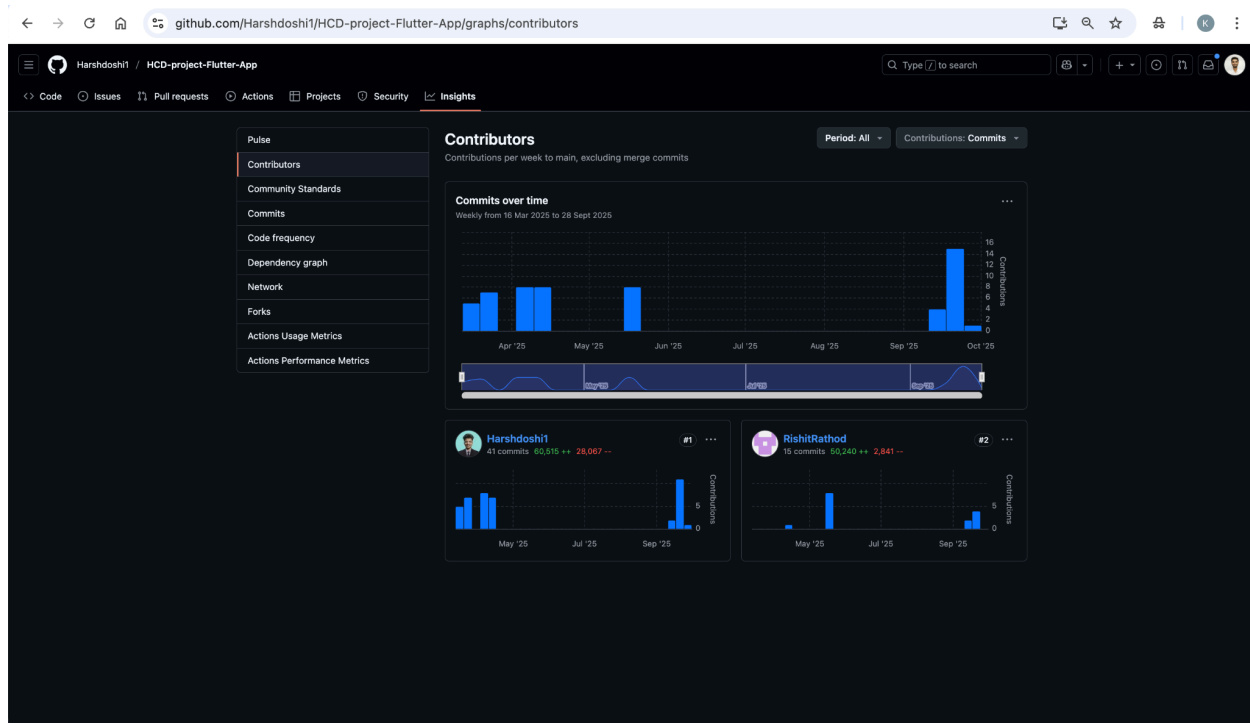
The screenshot shows the GitHub repository page for 'HCD-project' by user 'Harshdoshi1'. The repository is public and has 17 branches and 0 tags. A recent pull request #44 by 'krish26mamora' is highlighted. The repository description is: 'A role-based web portal for HODs and faculty to manage student performance, combining academic grades and extracurricular activities with automated scoring, subject allocation, and report generation, built with React.js, Node.js, Express.js, MySQL...'. The repository includes a README, a .DS_Store file, and several JavaScript files for testing and data handling. The 'About' section mentions 'hcd-project.vercel.app'.





Github repository , contributions and commit history for flutter app Repository





Integration Across Components

1. Database Integration : ORM-Based Data Access Model Definitions example

Student Model (/backend/models/students.js)

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/db');

const Student = sequelize.define('Student', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  enrollmentNumber: {
    type: DataTypes.STRING(20),
    allowNull: false,
    unique: true
  },
  firstName: {
    type: DataTypes.STRING(50),
    allowNull: false
  },
  lastName: {
    type: DataTypes.STRING(50),
    allowNull: false
  }
}, {
  // Additional options for the model
});
```

```
email: {
  type: DataTypes.STRING(100),
  allowNull: false,
  unique: true,
  validate: {
    isEmail: true
  }
},
batchId: {
  type: DataTypes.INTEGER,
  allowNull: false,
  references: {
    model: 'Batches',
    key: 'id'
  }
},
// Other student fields...
}, {
  timestamps: true,
  tableName: 'Students',
  indexes: [
    {
      unique: true,
      fields: ['enrollmentNumber']
    },
    {
      fields: ['batchId']
    }
  ]
}
]);

module.exports = Student;
```

2. API Integration Points : RESTful API Architecture

30+ endpoints are exposed and organized by domain.

```
app.use('/api/students', studentRoutes);
app.use('/api/student-marks', studentMarksRoutes);
app.use('/api/student-analysis', studentAnalysisRoutes);
app.use('/api/blooms-taxonomy', bloomsTaxonomyRoutes);
app.use('/api/course-outcomes', courseOutcomeRoutes);
app.use('/api/events', eventsRoutes);
app.use('/api/faculties', facultyRoutes);
```

1 . Frontend Request:

```
const response = await
fetch(buildUrl(`/student-analysis/blooms/${enrollmentNumber}/${semester}`));
const bloomsData = await response.json();
```


2 .Backend Processing:

```
const getBloomsTaxonomyDistribution = async (req, res) => {
  try {
    const { enrollmentNumber, semesterNumber } = req.params;

    const marks = await StudentMarks.findAll({
      include: [
        { model: ComponentWeightage, as: 'ComponentWeightage' },
        { model: SubjectComponentCo, as: 'SubjectComponentCos' }
      ],
      where: { enrollmentNumber, semesterNumber }
    });

    res.json(processedBloomsData);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

3. Authentication & Authorization Integration

Role-Based Access Control

```
<Route
  path="/admin/*"
  element={
    <ProtectedRoute allowRoles={['HOD']}>
      <DashboardHOD />
    </ProtectedRoute>
  }
/>
```

4 .Session Management

```
const ProtectedRoute = ({ children, allowRoles }) => {
  const userRole = localStorage.getItem('userRole');

  if (!allowRoles.includes(userRole)) {
    return <Navigate to="/login" replace />;
  }
}
```

```
return children;  
};
```

Component Interaction Flow

Data Flow Diagram:

